Operating Systems Restaurant

Alessio Zanon
Lorenzo Maduro

August 27, 2026

Contents

1 Description

The Restaurant itself is only responsible for the preparation and dispatching of threads. There are three thread types:

- Cooks
- Waiters
- Customers

Cooks receive dishes to prepare by reading a one-way many-to-one pipe, and upon completion of the dish write to the one-way many-to-one pipe with the waiter as receiver. Each Cook is assigned an atomic integer, shared with all waiters, that is used to keep track of their queue.

Waiters are notified of a customer arrival via a one-way many-to-many pipe, then read the dishes involved in their clients' orders from a shared allocated memory. The Waiter then chooses the best Cook for each dish, and sends the relevant information via one-way pipe. When not handling customer arrivals, waiters poll their assigned Cooks-to-Waiter pipe and attemp to entratain the Customer with lowest patience.

Customers generete their order randomly and place it in one slot of the shared memory with the waiters, then announce their slot's position in the pipe. Customers repeatedly poll their order completion and wait time, and modify the Restaurant's score when leaving.

2 Design Choices

2.1 Randomization

The seed taken from .env is used by the Splitmix64 library to generate sub-seeds by Restaurant. A separate set of sub-seeds are given to thread, and are then used with a modified xoshiro256plusplus to generate random numbers in each predictably manner. This way, changing the seed parameter in .env allows for the powerful splitmix64 library to do the heavy lifting and introduce entropy in the sub-seeds, and the independence of the sub seeds removes the possibility of a race condition between value generations, keeping the simulation predictable.

2.2 Extra-thread Communication

Comunication among threads is handled in two ways: Shared Memory and one-way Pipes

Shared Memory is used for all data that might need to be read or modified multiple times, like the "run" boolean for waiters and cooks, the order array, the cook's queue time array, and the client status.

When data instead was inteded as single-use, Pipes were chosen instead. Their main benefit was the inherent thread-safety and consistency they provide, as their operations are atomic (within the bounds of some sizeable bit amount) and the removal-on-read guarantees that even on -to-many pipes only one thread will read any one message. This way, the operating system shoulders the burden to guarantee that only one waiter attends any client, and that mutliple cook-to-waiter or waiter-to-cook messages don't collide and are instead resolved orderly by each thread.